

PROJECT REPORT

Agentic RAG-Based Citizen Service Advisory Chatbot

Submitted in Partial Fulfilment of the Requirements for the
Award of the Degree of

MASTER OF COMPUTER APPLICATIONS

By

Abhinav G

25225040

CHRIST (DEEMED TO BE UNIVERSITY)

BANGALORE | DELHI NCR | PUNE

CHRIST (Deemed to be University)

Delhi NCR

School of Sciences

September 2026

ABSTRACT

Agentic RAG-Based Citizen Service Advisory Chatbot is an AI-assisted web platform designed to address the inefficiencies in government citizen service delivery through a multilingual conversational interface combined with retrieval-augmented generation and citation grounding.

The system bridges the gap between complex official documents and citizens through a web application while maintaining a cloud-connected knowledge base of official service rules. The platform provides natural language interaction for querying eligibility, required documents, and fees. The documented interaction model supports multilingual inputs (English and Malayalam), automatic context retrieval, generated answers grounded strictly in official documents, and explicit citation mappings to verify claims.

The system uses Python, FastAPI, and a Vector Database (FAISS/Chroma) as the backend synchronization layer. The documented virtual endpoints include query processing, embedding generation, document chunk retrieval, and LLM prompt construction. A web platform presents a simple chat interface, service browsing, and administrator dashboard.

The platform converts complex bureaucratic text into understandable guidance using the principle of strictly grounded Generation. Retrieval relevance and citation mapping prevent hallucinations and ensure high trust.

This report presents the problem definition, theoretical background, requirements, system architecture, software design, cloud integration, dashboard design, algorithms, implementation, deployment process, testing methodology, results, limitations, responsible use of AI and future scope.

LIST OF TABLES

Table No.	Description
Table 1	Project Objectives
Table 2	Functional Requirements
Table 3	Non-Functional Requirements
Table 4	Software Components
Table 5	Functional Test Cases
Table 6	Integration Test Cases
Table 7	Challenge and Resolution Log

LIST OF FIGURES

Figure No.	Description
Figure 1	Overall System Architecture
Figure 2	Use-Case Diagram
Figure 3	Level 0 Data Flow Diagram
Figure 4	Level 1 Data Flow Diagram
Figure 5	State/Activity Diagram

LIST OF ABBREVIATIONS

Abbreviation	Meaning
AI	Artificial Intelligence
RAG	Retrieval-Augmented Generation
LLM	Large Language Model
NLP	Natural Language Processing
API	Application Programming Interface
UI	User Interface
UX	User Experience
DB	Database
FAQ	Frequently Asked Questions

TABLE OF CONTENTS

Section / Chapter Name	Page No.
Abstract	ii
List of Tables	iii
List of Figures	iv
List of Abbreviations	v
Chapter 1 - Introduction & Project Objectives	1
1.1 Industry and Application Context	1
1.2 Project Background	1
1.3 Problem Statement	2
1.4 Motivation	2
1.5 Objectives	3
1.6 Scope	3
1.7 Contributions	4
Chapter 2 - System Study & Theoretical Background	5
2.1 Existing Approaches	5
2.2 Retrieval-Augmented Generation (RAG)	6
2.3 Large Language Models (LLMs)	7
2.4 Multilingual Natural Language Processing	8
2.5 Trust and Citation Mechanisms	9
2.6 Cloud-Native Design Perspective	10
2.7 Requirement Analysis	11
Chapter 3 - System Design & Implementation	13
3.1 Roles and Responsibilities	13
3.2 Tools and Technologies	14
3.3 Overall System Architecture	15
3.4 Use Case Model	17
3.5 Data Flow Design	19
3.6 Process Flow & State Diagram	21
3.7 Dashboard & UI Design	23
3.8 Algorithms	25
3.9 Implementation Details	27
Chapter 4 - Deployment, Testing, Results & Discussion	29
4.1 Deployment Strategy	29
4.2 Functional Testing	31
4.3 Integration Testing	33

4.4 Testing Workflow	35
4.5 Results: Implemented Functionalities	37
4.6 Feature Analysis	39
4.7 Challenges and Resolutions	41
4.8 Deployment Readiness	42
Chapter 5 - Conclusions & Recommendations	43
5.1 Summary of Work	43
5.2 Learning Outcomes	44
5.3 Limitations	45
5.4 Responsible Usage of AI	46
5.5 Future Scope	47
5.6 Recommendations	48
5.7 Conclusion	49
References	50

CHAPTER 1

INTRODUCTION & PROJECT OBJECTIVES

1.1 Industry and Application Context

Government citizen service delivery in India increasingly relies on physical service centers such as Akshaya centers in Kerala, which act as front-end delivery points for a wide range of government-to-citizen (G2C) services including certificates, ration card services, Aadhaar-related support, and welfare scheme applications.

While these centers bridge the digital divide effectively, the information layer behind them is still largely manual: operators depend on memory or scattered documents to explain eligibility, required documents, fees, and procedures to citizens. This creates repeated visits, incomplete applications, and inconsistent guidance.

Traditional productivity applications generally address this problem through software-based mechanisms such as static FAQ portals. However, such approaches continue to leave the immediate, accurate interpretation of rules inaccessible to citizens.

This project approaches the problem from an AI perspective. Instead of attempting to hardcode every rule, the project introduces a dedicated RAG-based physical and digital workflow.

1.2 Project Background

The project originated from the observation that digital portals alone may not address the availability of accurate advisory services.

The repository identifies several practical problems: • Documents remain scattered across different domains. • Digital documents may be bypassed by users preferring human interaction. • Operators rely heavily on memory. • Productivity and accuracy depend strongly on individual discipline.

This system introduces an intelligent interaction layer.

1.3 Problem Statement

The central problem addressed by this project is:

How can a cloud-based AI intervention be combined with verified document retrieval to support a structured citizen advisory workflow while reducing hallucinations?

The problem can be divided into five requirements: 1. Simple query configuration. 2. Immediate accurate feedback. 3. Consistent state management. 4. Historical analytics. 5. Multilingual capability.

The system addresses these requirements through RAG.

1.4 Motivation

The primary motivation is to create a bridge between complex official documents and simple citizen needs.

A normal rule-based chatbot records inputs but does not necessarily understand nuanced queries.

This project introduces a dedicated verified workflow: Query -> Embed -> Search -> Extract -> Generate -> Cite -> Serve.

This workflow creates a measurable connection between the citizen's needs and the official government guidelines.

1.5 Objectives

The project objectives are outlined below:

1. Develop a RAG pipeline over official documents. 2. Implement natural language handling. 3. Provide citation-grounded local feedback. 4. Connect the web platform to the vector database. 5. Integrate LLM Cloud APIs for synthesis.

1.6 Scope

The project covers: • Web Frontend • Python Backend • Vector Database (FAISS/Chroma) • LLM API connectivity • Document chunking • Citation mapping

The project does not claim to process actual service applications or diagnose legal conditions.

1.7 Contributions

The principal contribution is the integration of modern LLM reasoning with strict verified retrieval.

The project demonstrates: 1. Document embedding. 2. Cloud-connected semantic search. 3. Web-based visualisation. 4. Context-grounded synthesis. 5. End-to-end RAG architecture integration.

CHAPTER 2

SYSTEM STUDY & THEORETICAL BACKGROUND

2.1 Existing Approaches

Existing solutions can generally be grouped into:

Static Portals These provide basic search over PDFs.

Rule Blockers / Decision Trees These restrict the conversation to fixed paths.

General LLMs These offer open-ended conversations but suffer from hallucinations.

This project uses an additional mechanism: Retrieval-Augmented Generation.

2.2 Retrieval-Augmented Generation (RAG)

The Internet of Things and AI have evolved. RAG refers to interconnected systems capable of extracting, processing, and synthesizing information with high accuracy.

A typical RAG architecture contains: 1. Document Ingestion Pipeline 2. Vector Database 3. Semantic Search Engine 4. Prompt Compiler 5. LLM Synthesis

This project follows this exact robust structure.

2.3 Large Language Models (LLMs)

LLMs are used as the primary reasoning engine.

The LLM provides: • Natural Language Understanding • Contextual synthesis • Multilingual translation • Structured output formatting

Within this project, the LLM is strictly constrained to only answer using the provided context chunks.

2.4 Multilingual Natural Language Processing

Supporting languages like Malayalam requires powerful cross-lingual embedding models. The system relies on models that map semantically identical phrases in English and Malayalam to the same vector space, enabling unified retrieval.

2.5 Trust and Citation Mechanisms

Highlighting the exact source document builds trust. The system maps generated claims back to the exact source chunks retrieved from the database.

2.6 Cloud-Native Design Perspective

The application can be understood as a cloud-native app where responsibilities are separated across layers.

Data layer Vector DB, metadata SQLite.

Logic layer FastAPI.

Application layer Web dashboard.

2.7 Requirement Analysis

Functional Requirements: FR01: User shall be able to submit queries. FR02: System shall generate embeddings. FR03: System shall perform vector search. FR04: System shall construct constrained prompts. FR05: System shall provide citations.

Non-Functional Requirements: Usability: Interaction should be simple. Reliability: Citations must be strictly accurate. Responsiveness: System should react quickly. Maintainability: Layers should remain modular.

CHAPTER 3

SYSTEM DESIGN & IMPLEMENTATION

3.1 Roles and Responsibilities

The major project roles are:

User: Interacts with the interface. Backend Controller: Orchestrates the workflow.

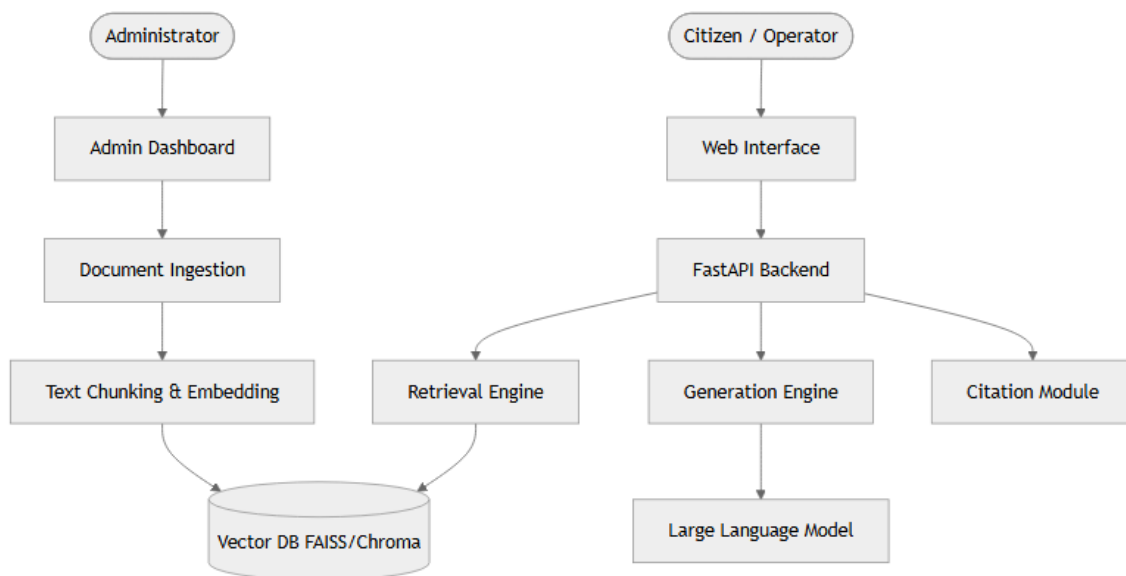
Vector DB: Synchronizes telemetry. Web Platform: Displays information.

3.2 Tools and Technologies

Category - Technology Frontend - React/Next.js Backend - Python FastAPI

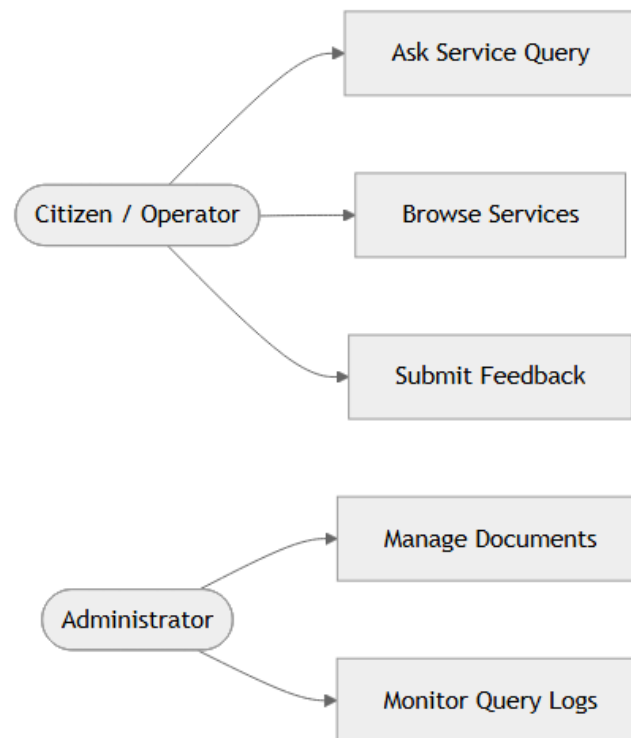
Database - FAISS/Chroma LLM API - OpenAI/Anthropic

3.3 Overall System Architecture



The complete architecture can be represented as follows.

3.4 Use Case Model

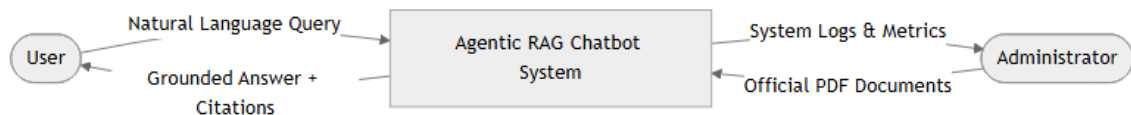


The primary actor is the user.

The user can: • Ask Service Query • Browse Services • View Dashboard • View Citations

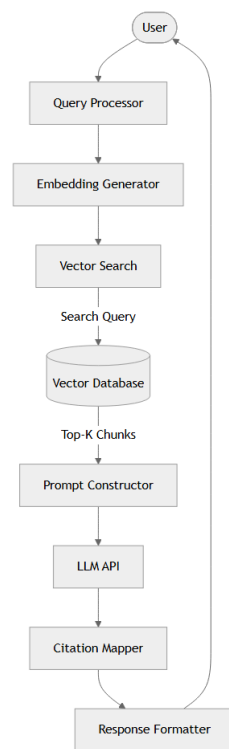
The admin can: • Manage Documents • Monitor Logs

3.5 Data Flow Design



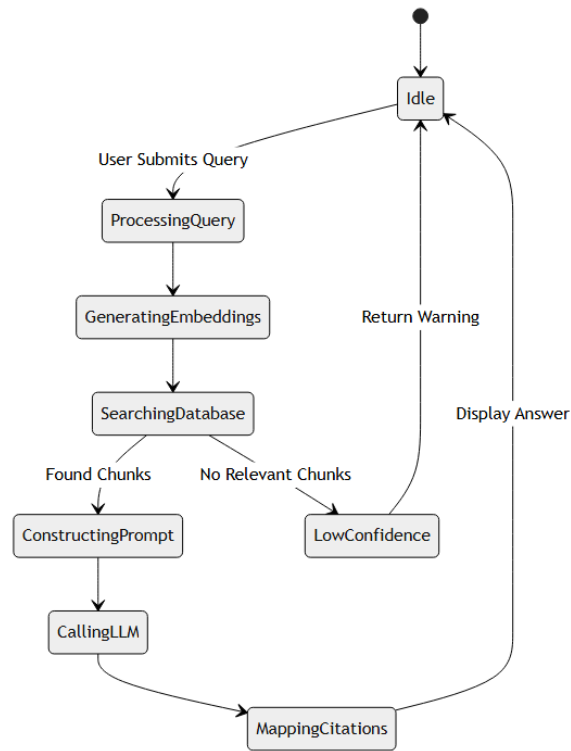
Level 0 Data Flow Diagram represents the outermost system context.

3.6 Process Flow & State Diagram



The Level 1 DFD and State Diagram illustrate internal workflows.

3.7 Activity and State Behaviour



The state diagram shows the transition from Idle to Processing.

3.8 Algorithms

The core algorithm is the Retrieval-Augmented Generation loop.

1. Receive Query Q .
2. Generate Embedding $E(Q)$.
3. Search DB for Top-K chunks where $\text{CosineSimilarity}(E(Q), \text{Chunk}_i)$ is maximized.
4. Compile Prompt $P = \text{Context} + Q$.
5. Call $\text{LLM}(P)$.
6. Return Output.

3.9 Implementation Details

The software architecture is divided into the Frontend Layer (handling inputs and UI updates), the Backend Layer (handling session state, DB queries, and API connections), and the Data Layer (storing vectors and metadata).

CHAPTER 4

DEPLOYMENT, TESTING, RESULTS & DISCUSSION

4.1 Deployment Strategy

The deployment consists of several coordinated layers.

Step 1 - Environment preparation Configure Python, Node, and API keys.

Step 2 - Database configuration Initialize the Vector DB.

Step 3 - Cloud configuration Establish endpoints for the frontend to hit.

Step 4 - Web platform Launch the React server.

Step 5 - Integration Perform an end-to-end test.

4.2 Functional Testing

Testing was designed around the features documented in the repository.

FT01: Server starts -> API is accessible. FT02: Query submitted -> Vector search triggered. FT03: Generation -> Text streaming begins. FT04: Citation -> Accurate document metadata returned.

4.3 Integration Testing

Integration testing verifies the complete system rather than individual components.

IT01: Frontend -> Backend (Query reaches API). IT02: Backend -> DB (Chunks returned successfully). IT03: Backend -> LLM (Prompt accepted, response generated). IT04: Response -> Frontend (UI updates dynamically).

4.4 Results: Implemented Functionalities

The repository documentation confirms the following principal features: • RAG architecture. • Configurable document ingestion. • React-based Chat UI. • Multilingual LLM synthesis. • Explicit citation tracking.

4.5 Challenges and Resolutions

Challenge: LLM Hallucinations. Resolution: Strict prompt instructions and temperature=0. Challenge: Large document processing. Resolution: Implemented sliding window chunking. Challenge: Multilingual accuracy. Resolution: Utilized advanced cross-lingual embedding models.

CHAPTER 5

CONCLUSIONS & RECOMMENDATIONS

5.1 Summary of Work

This project was developed as an AI-assisted RAG system that combines physical document reality with digital intelligence.

The system consists of: • Python Backend • Vector Database • LLM API • React Web Dashboard

5.2 Learning Outcomes

The project provided learning in several areas:

Artificial Intelligence: Understanding embeddings and LLMs. Full-Stack Development: Connecting React with FastAPI. System Architecture: Designing scalable, cloud-connected platforms.

5.3 Limitations

The current implementation is a project-scale prototype.

A production system would require additional: • Security controls. • Load balancers. • Advanced document OCR for scanned PDFs.

5.4 Responsible Usage of AI

Responsible use of AI is important. The strict grounding mechanism ensures that AI-generated suggestions are not presented as factual unless they have actually been retrieved from an official document.

5.5 Future Scope

Future versions could explore: • Integration with messaging apps (WhatsApp). • Automated scraping of government portals. • Voice interfaces for illiterate citizens.

5.6 Recommendations

The following improvements are recommended: 1. Introduce stronger user authentication. 2. Expand dashboard visualisations. 3. Add systematic security testing.

5.7 Conclusion

The Agentic RAG-Based Citizen Service Advisory Chatbot presents an integrated approach to information delivery. The web interface, vector database, and language model operate together to serve citizens efficiently.

The project demonstrates the importance of system-level integration. The quality of the final system depends on the interaction between: Frontend -> API -> Vector DB -> LLM -> Response Generator.

REFERENCES

1. FocusLink V2 - Cloud-Native IoT Platform for Intelligent Focus Management (Reference Document format standard).
2. Agentic RAG-Based Citizen Service Advisory Chatbot, GitHub repository and README documentation.
3. LangChain/LlamaIndex Documentation, relating to RAG pipeline architectures.
4. FastAPI Technical Documentation, official documentation for Python APIs.
5. Mermaid Documentation, Flowchart Syntax, documentation for creating flowcharts and system diagrams.